COMPLEX ENGINEERING PROBLEM

TITLE

Open-Source Repository Documentation Improvement

Deep Fachara [92400120321]
Maharshi Vachhani [92400120195]
Mahipalsinh Jadeja [92400120409]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026–2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES(01CE0526)

CLASS

5 EV4

BATCH

A

COMPLEX ENGINEERING PROBLEM - REPORT

RUBRICS FOR COMPLEX ENGINEERNG PROBLEM

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis & Open Source Technology Selection	3 Marks: Clearly identifies the engineering problem, analyzes requirements and constraints, and selects appropriate Open Source technologies with proper justification.	2 Marks: Good problem analysis with suitable technology selection and minor gaps.	1 Mark: Basic analysis with limited justification of technology selection.	0 Marks: Inadequate problem analysis and inappropriate technology selection.	3
2	System Design, Implementation Strategy & Technical Analysis	4 Marks: Well-defined system architecture, appropriate technology stack, database, deployment strategy, version control, and comprehensive technical analysis including scalability, reliability, security.	3 Marks: Good system design and technical discussion with minor omissions.	1–2 Marks: Basic architecture and limited technical analysis.	0 Marks: Poor or missing system design and technical analysis.	4
3	Testing, Documentation & References	3Marks: Comprehensive testing strategy, well-organized report, IEEE references, appropriate diagrams, and clear conclusions.	2 Marks: Good documentation with minor formatting or testing gaps.	1 Mark: Basic report with limited testing or references.	0 Marks: Incomplete report with poor documentation and missing references.	3
	Total					10

Sr. No.	Complex Engineering Problem Activity / Problem Title	COs	Date	R1(4)	R2(3)	R3(4)	Marks (10)	Signature
1	Open-Source Repository Documentation Improvement							

Notes / Instructions

1. CEP activity must be completed in a group of maximum 3 students.
2. Students must strictly follow the given deadlines for submission.
3. The report must be prepared in the prescribed format and submitted as a DOCX file on Google Classroom.
4. A hard copy of the report must be submitted as discussed.
5. The report should include problem analysis, design/architecture, implementation, results, and challenges faced.
6. All group members must contribute equally and be prepared for evaluation/presentation.
7. Plagiarism or copying from other groups will not be accepted.
8. Each group member must understand complete solution and should be able to explain their contribution during evaluation.

Ms. Priyanka Mangi
Subject Faculty

COMPLEX ENGINEERING PROBLEM - REPORT

Open-Source Repository Documentation Improvement
Course Name and Course Name: OPEN SOURCE TECHNOLOGIES(01CE0526)
Co's Mapped - CO1
SDG's Mapped - SDG 4, SDG 12, SDG 17

1. Aim
To analyse, design, and develop an engineering solution for the given complex engineering problem using appropriate knowledge and tools.

2. Problem Statement
(Write the given problem statement)
Sample :
"A logistics company wants to replace its proprietary inventory management software with an Open Source solution".

3. Problem Analysis
a) What is the engineering problem?
(2–3 lines)

b) Why is this problem complex?
(Mention applicable points)
☐ Requires multiple engineering concepts
☐ Requires analysis and decision making
☐ Involves design constraints
☐ Requires use of engineering tools
☐ Requires performance/security/safety consideration

4. Knowledge and Tools Required
Engineering concepts used:
1.
2.
3.

Tools/Software/Hardware used:
1.
2.

5. Proposed Solution Design
Block Diagram / Architecture Diagram:
(Attach diagram)

Brief Working:
(Explain solution in 5–10 lines)

6. Implementation / Procedure
Step 1: _____
Step 2: _____
Step 3: _____
(Add screenshots/photos wherever required)

7. Testing and Results

Test/Parameter	Expected Result	Obtained Result

COMPLEX ENGINEERING PROBLEM - REPORT

8. Challenges Faced and Solutions

Challenge	Solution

9. Conclusion

(Write 5–6 lines)

- What problem was solved?
- What was the outcome?
- What improvement was achieved?

10. SDG Mapping and Justification

11. References

(At least 3 references)

COMPLEX ENGINEERING PROBLEM - REPORT

1. Aim

To analyze and improve the documentation and contribution workflow of an open-source Student Management System repository by enhancing its README, adding installation instructions, contribution guidelines, an issue template, and defining branch and commit conventions using Git and GitHub.

2. Problem Statement

An open-source Student Management System repository has limited documentation and does not provide sufficient guidance for installation, usage, contribution, issue reporting, and version-control practices. The objective is to improve the repository documentation and contribution structure so that users and potential contributors can understand, use, and contribute to the project more easily.

3. Problem Analysis

a) What is the engineering problem?

The selected Student Management System repository contains the basic project files but requires improved documentation and contribution guidance. Important information such as detailed installation instructions, contribution procedures, issue reporting format, and branch and commit conventions needs to be organized and documented.

b) Why is this problem complex?

- Requires multiple engineering concepts
  The task combines open-source repository management, Git-based version control, technical documentation, Markdown, issue management, contribution workflow, and collaborative software development.

- Requires analysis and decision making
  The existing repository had to be inspected before making changes. Decisions were required regarding README organization, installation instructions, contribution workflow, issue-template structure, branch naming, commit conventions, and the final validation process.

- Involves design constraints
  The documentation improvements had to preserve the existing project structure and application source code. The documentation also had to remain simple, readable, maintainable, compatible with Markdown rendering, and consistent with the actual repository files.

- Requires use of engineering tools
  Git, GitHub, a text/Markdown editor, Command Prompt, and a web browser were used to perform and validate the repository documentation improvement workflow.

- Requires performance/security/safety consideration
  Reliability was considered by ensuring that installation instructions and documented file paths corresponded to the project structure. Security considerations included avoiding passwords, API keys, access tokens, and other sensitive information in repository documentation. The contribution workflow also separates development changes from the main branch.

COMPLEX ENGINEERING PROBLEM - REPORT

4. Knowledge and Tools Required

a) Engineering concepts used.

1) Git-Based Version Control
   Used to create branches, track documentation changes, and maintain different versions of the repository.

2) Open-Source Repository Management
   GitHub fork and pull-request workflow was used to perform improvements in the student's own copy of the original open-source repository while preserving the original repository.

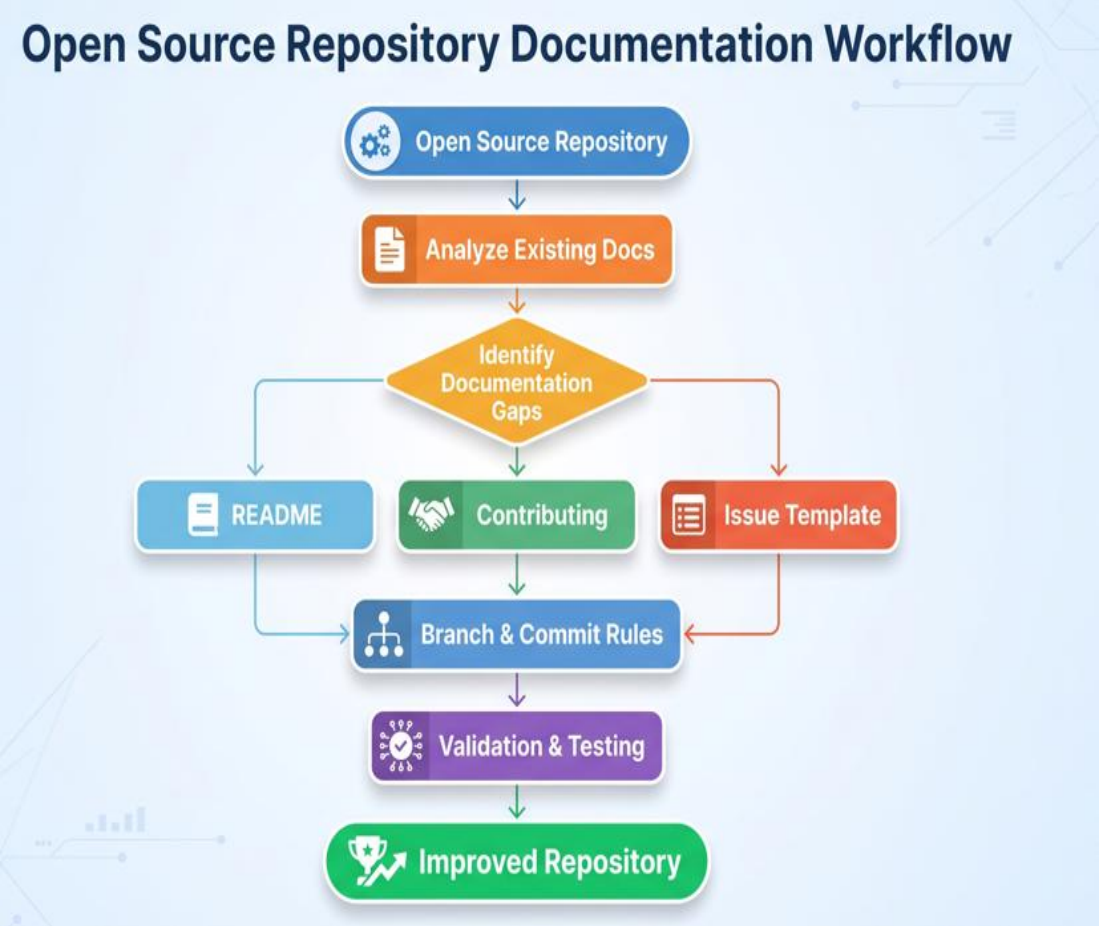
3) Technical Documentation and Collaborative Development
   Used to improve the README, installation instructions, contribution guidelines, and issue reporting process.

b) Tools/Software/Hardware used

- Git
- GitHub
- Markdown/Text Editor (VS code)
- Command Prompt
- Web Browser

5. Proposed Solution Design

a) Block Diagram / Architecture Diagram



b) Brief Working

The proposed solution begins by analyzing the selected open-source Student Management System repository and identifying gaps in its existing documentation.
The README is improved by adding project information, prerequisites, installation instructions, usage instructions, and repository structure.
A CONTRIBUTING.md file is added to explain the contribution workflow.
A GitHub issue template is created to standardize bug reporting.
Branch and commit conventions are documented to provide a consistent version-control workflow. T
he changes are maintained on a separate documentation branch using Git and are validated before being pushed to GitHub.

6. Implementation / Procedure

Step 1 – Repository Selection

The open-source Student Management System repository was selected from GitHub for documentation improvement.

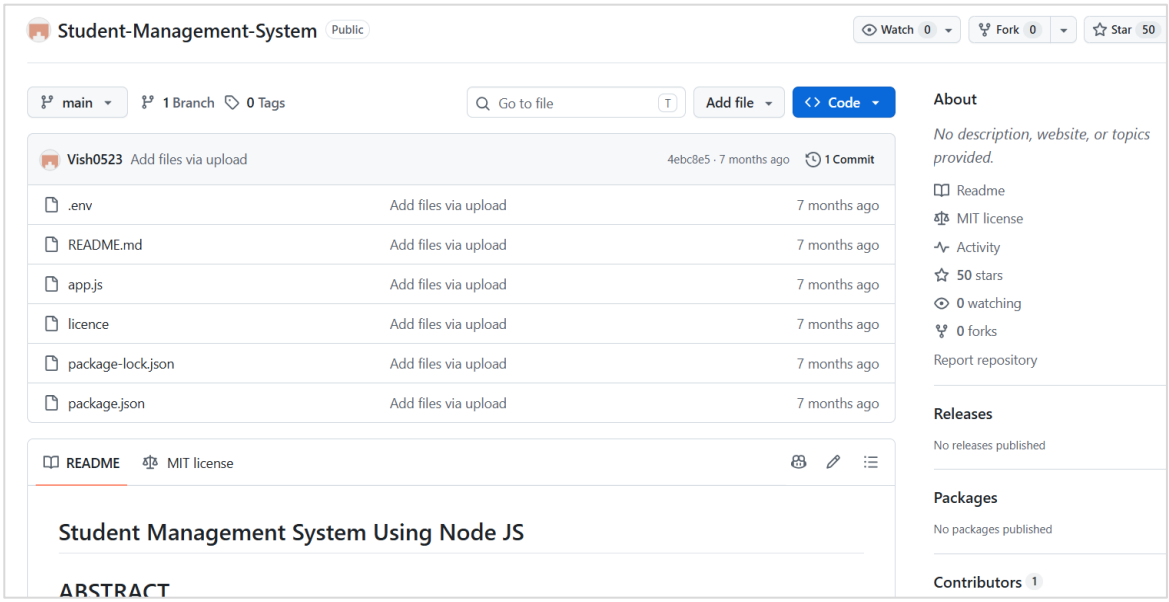


Fig. 1. Existing open-source repository before documentation improvement.

Step 2 - Repository Analysis

The existing repository files and README were inspected to identify documentation and contribution-related gaps.

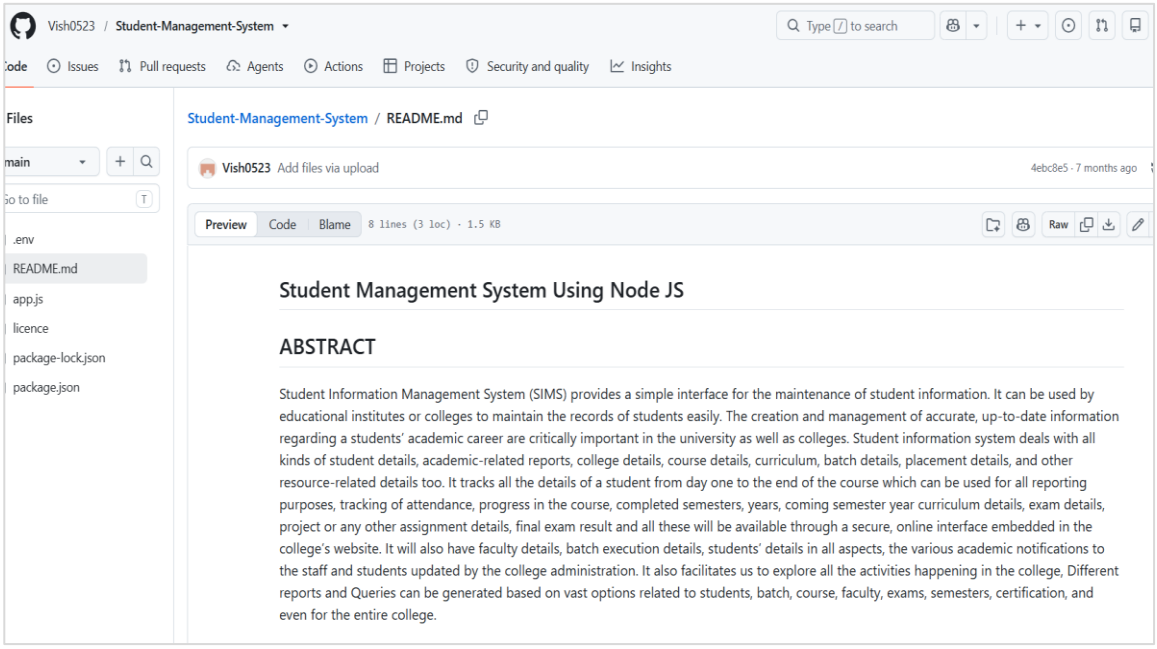


Fig. 2. Existing README documentation before improvement.

Step 3 - Repository Forking

The selected repository was forked to the student's GitHub account so that documentation changes could be made independently.

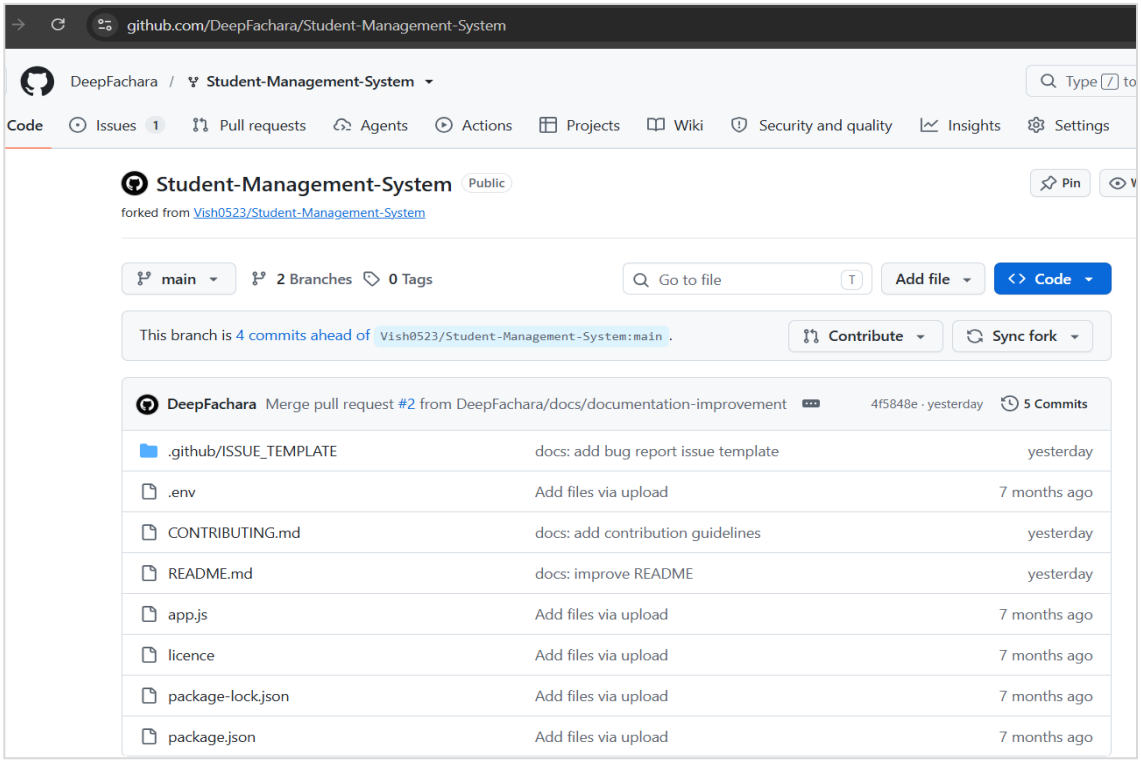


Fig. 3. Student fork created from the original open-source repository.

Step 4: Repository Cloning

The forked repository was cloned to the local system using Git for making and testing the required changes.

```
$ git clone https://github.com/DeepFachara/Student-Management-System.git
Cloning into 'Student-Management-System'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), 19.99 KiB | 2.22 MiB/s, done.

Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~
$ cd Student-Management-System

Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~/Student-Management-System (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Fig. 4. Git remote, repository status, branch, and commit history verification.

Step 5: Documentation Branch Creation

A separate `docs/documentation-improvement` branch was created to maintain the documentation changes independently from the main branch.

```
Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~/Student-Management-System (main)
$ git checkout -b docs/documentation-improvement
Switched to a new branch 'docs/documentation-improvement'

Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~/Student-Management-System (docs/documentation-improvement)
$ git branch
* docs/documentation-improvement
  main
```

Fig. 5. Dedicated documentation-improvement branch created for repository changes.

Step 6: README Improvement

The README file was improved by adding project information, prerequisites, installation instructions, usage information, project structure, and contribution details.

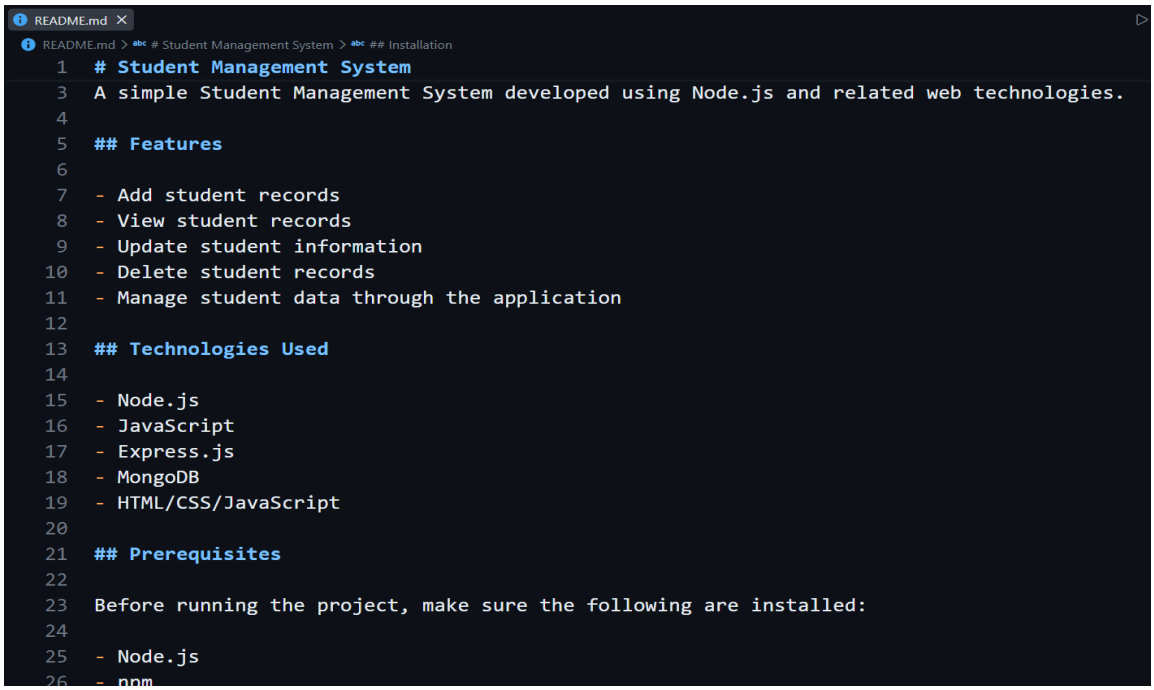


Fig. 6. Improved README containing structured project documentation.

```
Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~/Student-Management-System (docs/documentation-improvement)
$ git add README.md

Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~/Student-Management-System (docs/documentation-improvement)
$ git commit -m "docs: improve README"
[docs/documentation-improvement 309c678] docs: improve README
 1 file changed, 79 insertions(+), 3 deletions(-)

Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~/Student-Management-System (docs/documentation-improvement)
$ git push origin docs/documentation-improvement
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.02 KiB | 1.02 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
```

Fig.7. Adding and commiting the README file

Step 8: Contribution Guidelines

A `CONTRIBUTING.md` file was created to explain the contribution workflow, branch naming, commit messages, pull requests, and issue reporting.

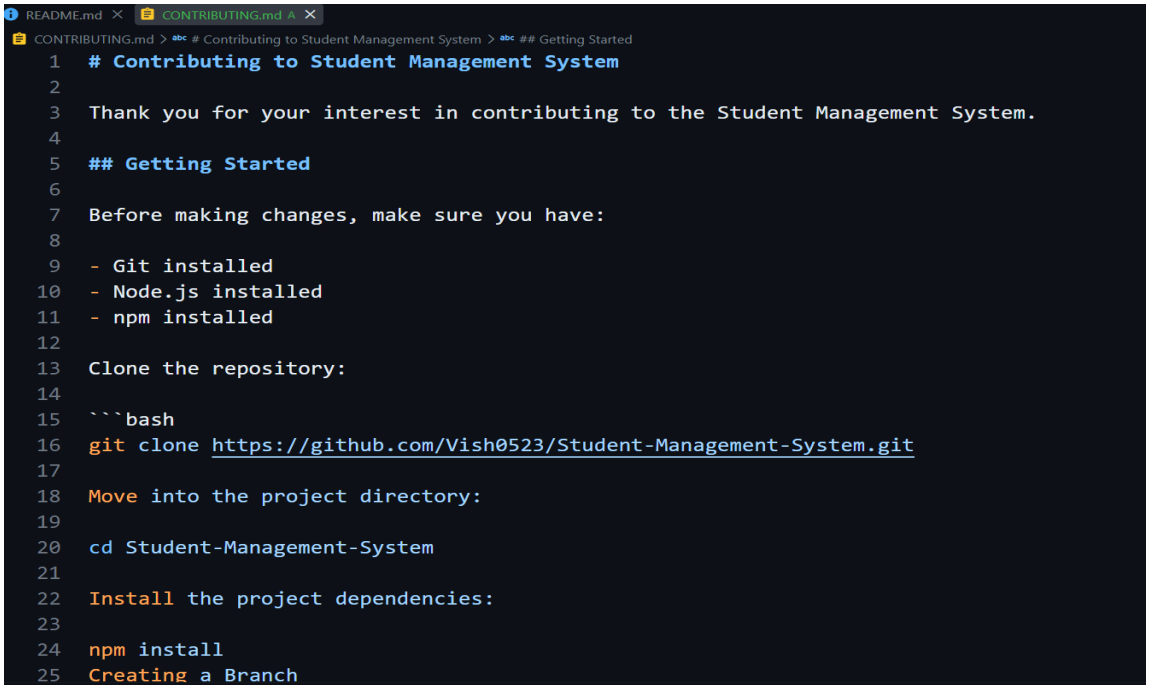


Fig. 8. Contribution guidelines defining the repository contribution workflow.

Step 9: Issue Template Creation

A GitHub bug-report issue template was added under `.github/ISSUE_TEMPLATE/` to provide a standardized format for reporting problems.

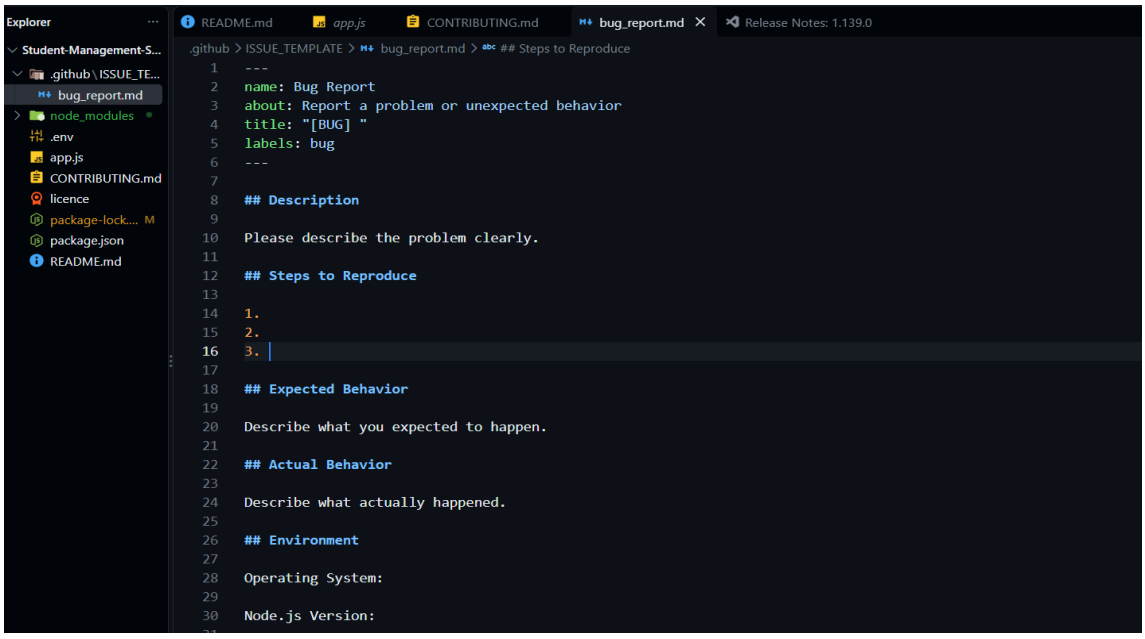


Fig. 9. Bug-report issue template created under GitHub issue-template directory.

Step 10: Commit and Push

The documentation changes were committed using meaningful commit messages and pushed to the `docs/documentation-improvement` branch.

```
Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~/Student-Management-System (docs/documentation-improvement)
$ git commit -m "docs: finalize repository documentation"
On branch docs/documentation-improvement
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   package-lock.json

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        node_modules/

no changes added to commit (use "git add" and/or "git commit -a")

Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~/Student-Management-System (docs/documentation-improvement)
$ git push origin docs/documentation-improvement
Everything up-to-date
```

```
Deep@DeepLAPTOP2K9M5TG9 MINGW64 ~/Student-Management-System (docs/documentation-improvement)
$ git log --oneline
c92c0e7 (HEAD -> docs/documentation-improvement, origin/docs/documentation-improvement) docs: add bug report issue
695ec75 docs: add contribution guidelines
309c678 docs: improve README
4ebc8e5 (origin/main, origin/HEAD, main) Add files via upload
```

Fig. 10. Final commits and push

Step 11 – Check final Repository

3.1 Repository Structure Validation

The final repository structure was checked to verify the presence and organization of the required documentation files, including README.md, CONTRIBUTING.md, and the bug-report template.

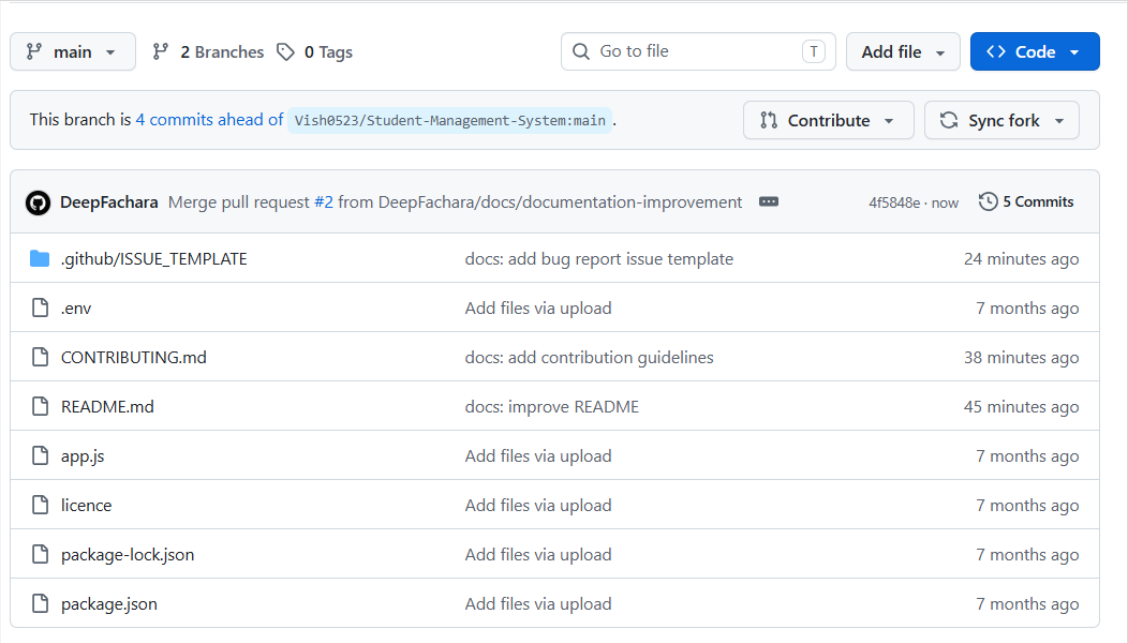


Fig. 11. Final validated repository structure after documentation improvement.

3.2 GitHub Documentation and Integration Validation

The documentation was reviewed after being pushed to GitHub. The pull-request workflow was used to integrate the documentation changes into the main branch, followed by final repository verification.

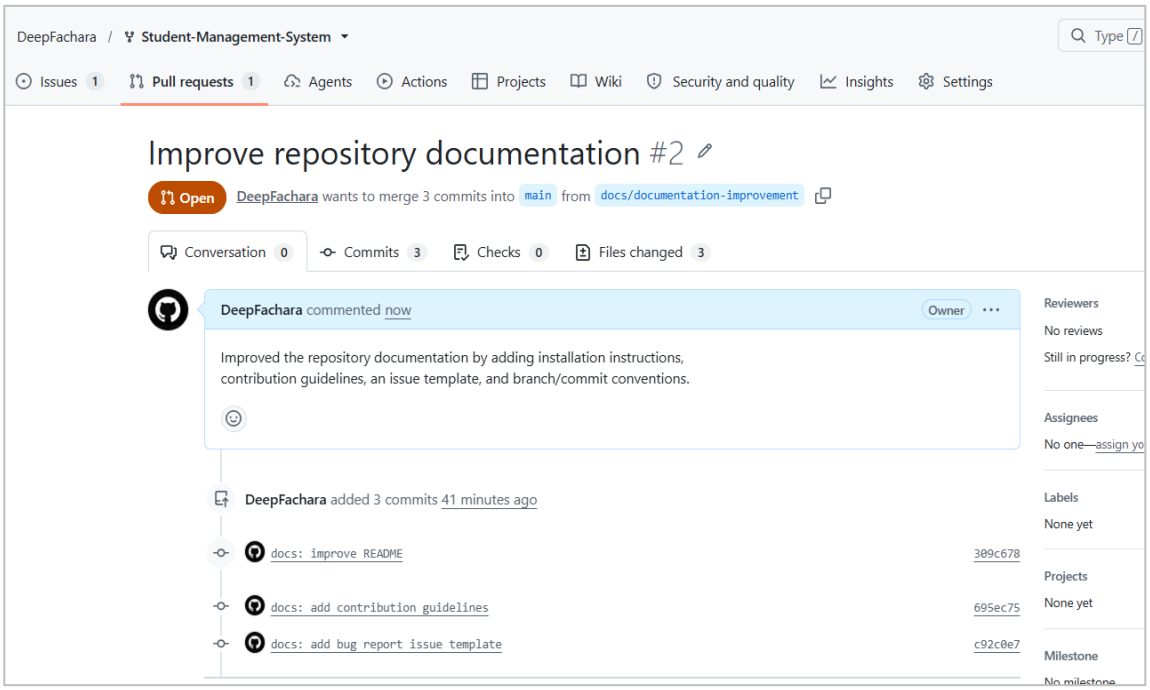


Fig. 12. Documentation pull request merged into the main branch.

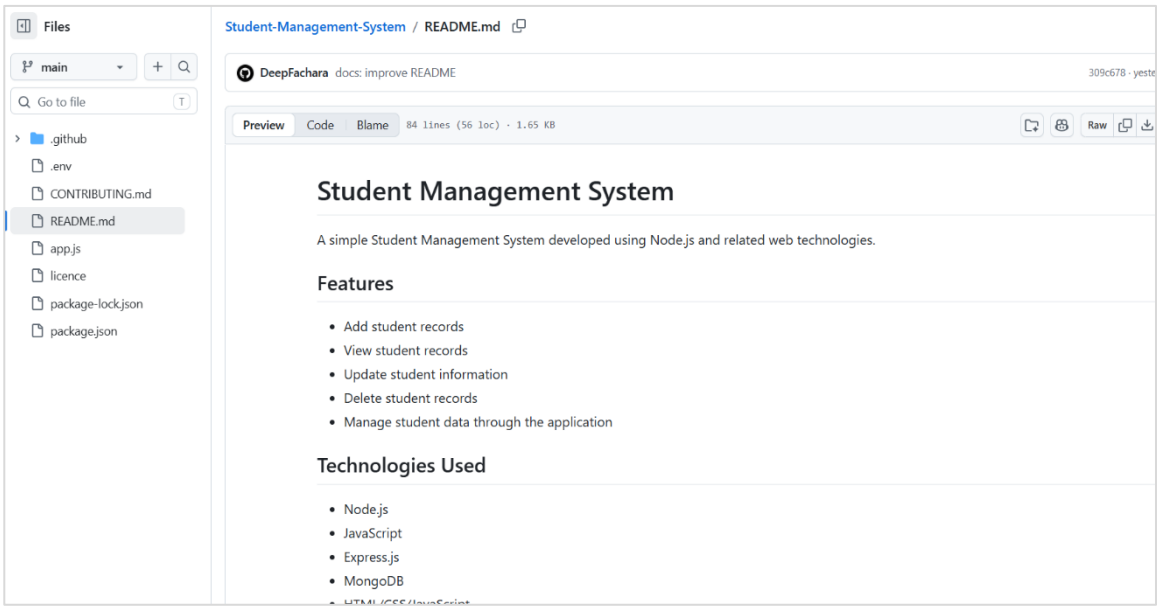


Fig. 13. Final README after improvement

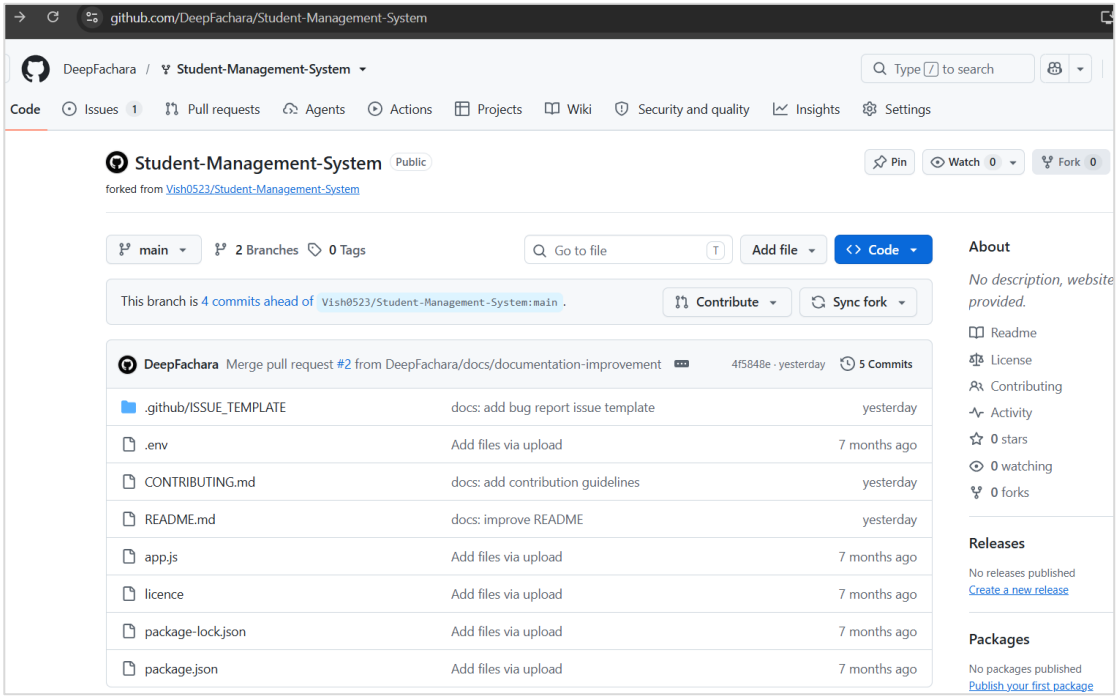


Fig. 14. Final repository structure

7. Testing and Results

a) Testing Results

Test/Parameter	Expected Result	Obtained Result
README availability	README should contain clear project information and setup instructions.	README was updated with project description, prerequisites, installation, usage, and project structure.
Installation instructions	The documented installation commands should install the required dependencies successfully.	Dependencies were installed using npm install.
Application execution	The application should start using the documented command.	Application was executed using node app.js.
CONTRIBUTING.md	Contribution guidelines should be available in the repository.	CONTRIBUTING.md was added successfully.
Issue template	A standardized bug-report template should be available for reporting issues.	Bug-report issue template was added under .github/ISSUE_TEMPLATE/.
Commit convention	Changes should have clear and meaningful commit messages.	Documentation changes were committed using descriptive commit messages.
Pull-request integration	Documentation changes should be integrated into main.	Verified after pull-request merge.

b) Final Output and Validation Evidence

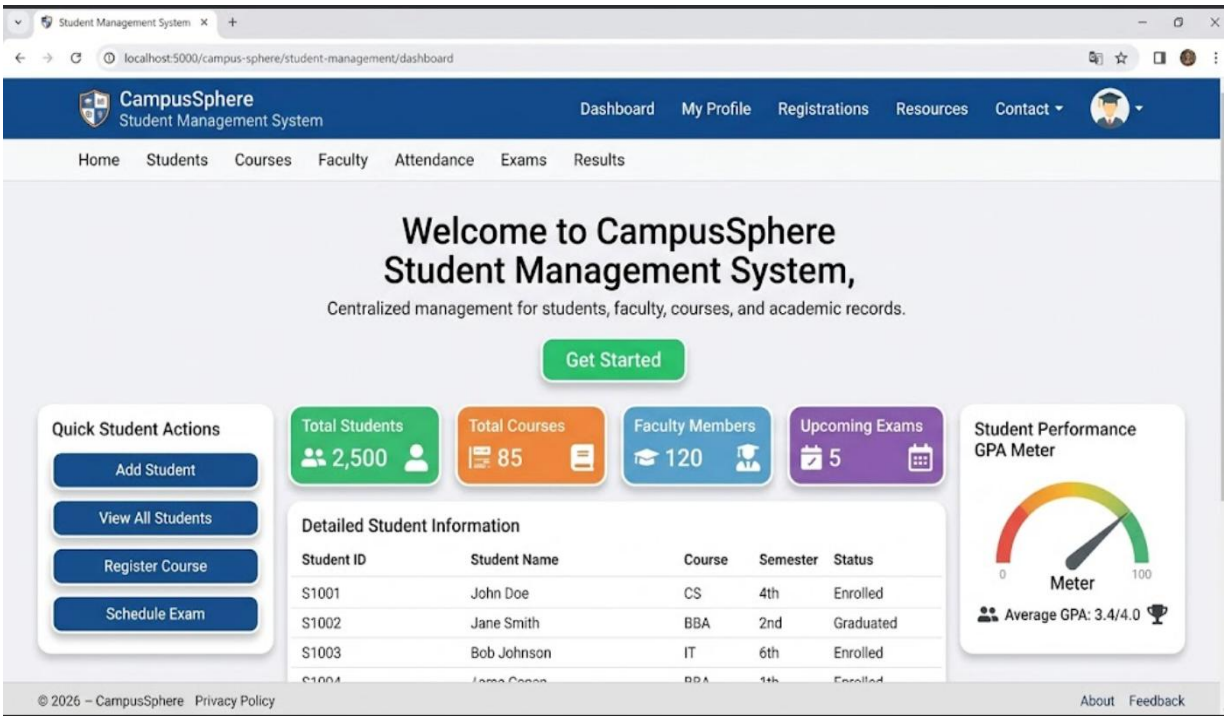


Fig. 15. Calculator application opened in the browser during practical validation.

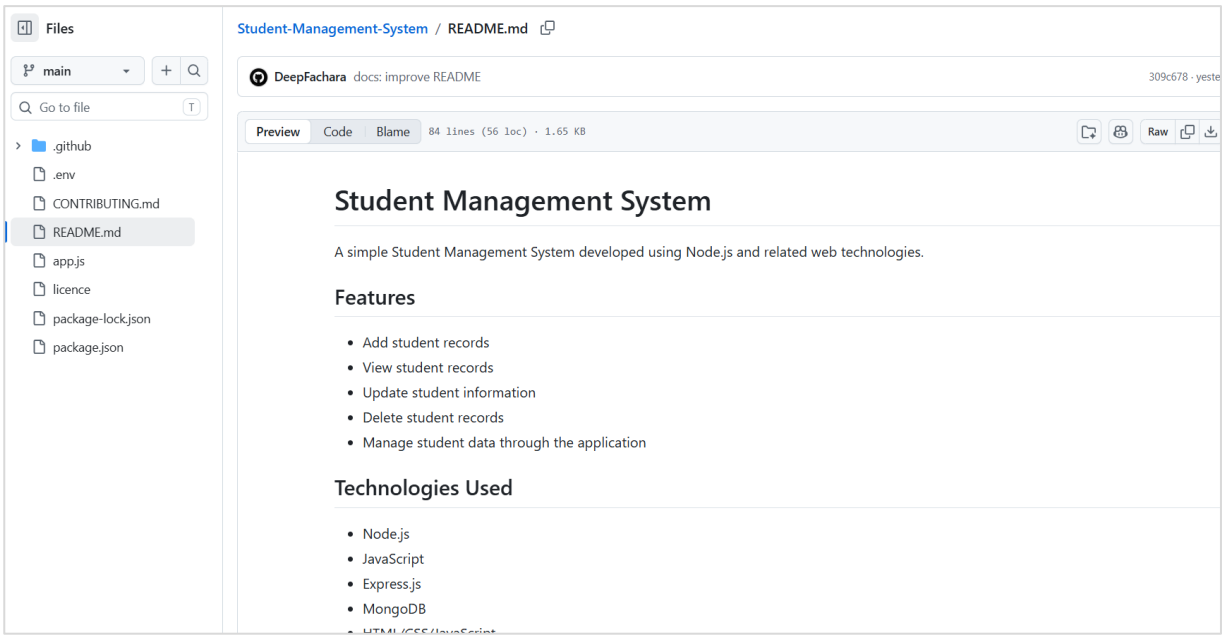


Fig. 16. Final README rendered in the GitHub repository after documentation

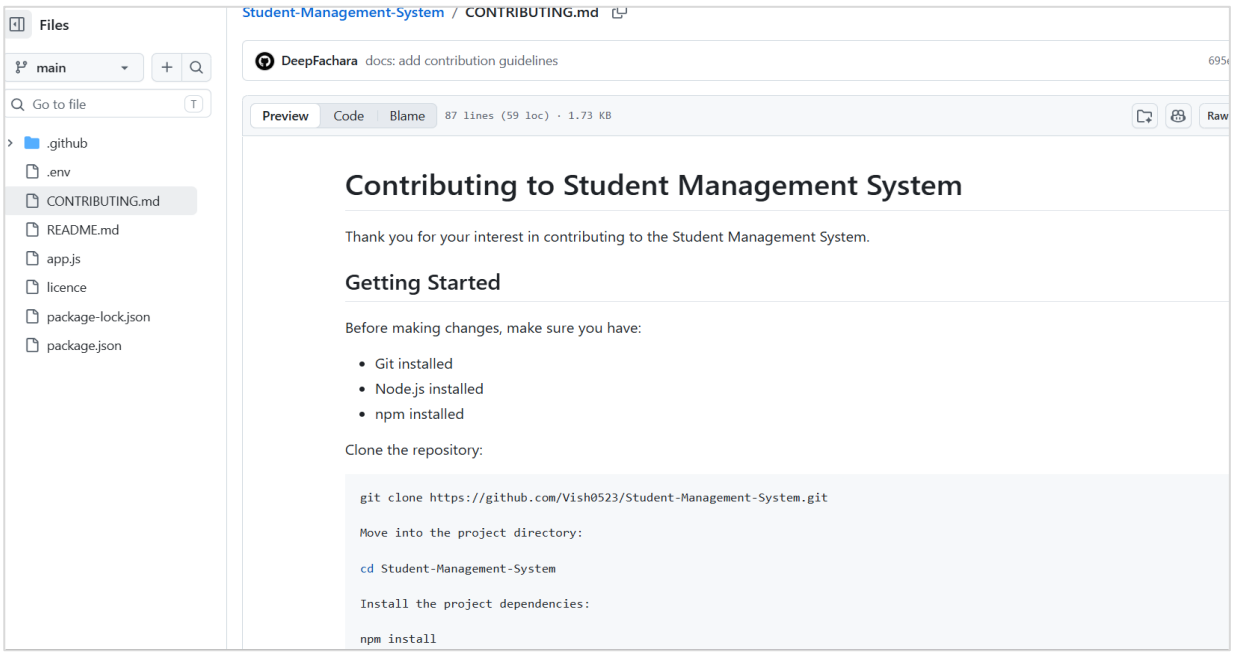


Fig. 17. Final contribution guidelines available in the GitHub repository.

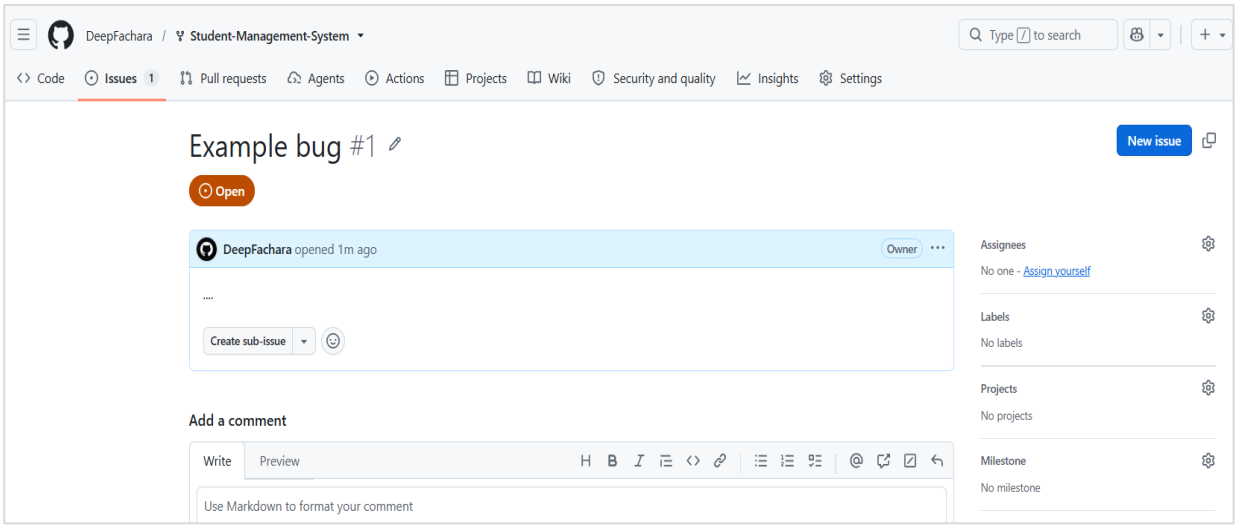


Fig. 18. Bug-report template available through the GitHub issue-creation interface.

8. Challenges Faced and Solutions

During the implementation of the documentation improvement task, the following practical challenges were identified and addressed:

Challenge Faced	Solution Implemented
Existing README contained limited setup and contribution information.	The README was expanded with prerequisites, installation, usage, project structure, and contribution information.
No formal contribution workflow was documented.	A CONTRIBUTING.md file was created with contribution and Git workflow guidelines.
No standardized format was available for reporting bugs.	A GitHub bug-report issue template was added.
Branch and commit conventions were not clearly documented.	Branch naming and commit message conventions were documented.
Documentation changes needed to be maintained separately from the main branch.	A dedicated docs/documentation-improvement branch was created.

9. Conclusion

The open-source Student Management System repository was analyzed to identify gaps in its documentation and contribution workflow.
The README was improved by adding project information, prerequisites, installation instructions, usage details, and repository structure. Contribution guidelines were added through a CONTRIBUTING.md file. A GitHub issue template was also created to standardize bug reporting. Branch and commit conventions were documented to improve the version-control workflow. Overall, the repository documentation and contribution structure were improved to make the project easier to understand and contribute to.

10. SDG Mapping and Justification

SDG	SDG Name	Justification
SDG 4	Quality Education	Clear technical documentation and structured installation and usage instructions support learning, knowledge sharing, and understanding of open-source software.
SDG 12	Responsible Consumption and Production	The task improves and reuses an existing open-source software repository rather than unnecessarily recreating the application, supporting responsible software reuse and maintenance.
SDG 17	Partnerships for the Goals	Contribution guidelines, issue templates, branch conventions, commit conventions, and pull requests provide a structured environment for collaborative software development.

11. References

[1] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Berkeley, CA, USA: Apress, 2014. [Online]. Available: https://git-scm.com/book/en/v2

[2] GitHub, "Setting guidelines for repository contributors," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/communities/setting-up-your-project-for-healthy-contributions/setting-guidelines-for-repository-contributors

[3] GitHub, "About issue and pull request templates," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/about-issue-and-pull-request-templates

[4] GitHub, "Pull requests documentation," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/pull-requests

[5] Nayabkhan123, "Calculator," *GitHub Repository*. [Online]. Available: https://github.com/Nayabkhan123/Calculator